Notes on Programming in SCCC

by
Ivan Gaydardzhiev

ABSTRACT

SCCC is a self-compiling C compiler for x86-64. It preprocesses and compiles C source, emits machine instructions, resolves addresses, and writes the executable itself. For Linux programs it supplies its own headers and C library and produces ELF64 files. For kernels it produces flat binaries, BIOS disk images, and Multiboot images, including the boot code needed to enter 64-bit mode. These notes describe the language, code generation, runtime conventions, hardware instructions, boot loaders, and the bootstrap in which the compiler compiles itself.

1. Introduction

The compiler accepts one or more C files and produces x86-64 machine code. For Linux programs it writes an ELF executable; for freestanding programs it can write a flat binary, a BIOS-bootable disk image, or a Multiboot image. No assembler or linker is invoked. Instruction encodings are appended to a byte buffer, and the final writer places the code and data at their runtime addresses.

A hosted compilation reads the program on the command line, its included files, the embedded headers, and the embedded implementation of the C library. The library source is compiled after the user files. With `-ffreestanding`, the compiler reads kernel source without inserting a C library or process startup. In both cases a single compilation context holds the text, data, symbols, and unresolved address references.

The implementation uses recursive descent for expressions, declarations, and statements. Types are constructed as declarations are parsed. Expression parsing also generates instructions, so the compiler does not retain a program-wide syntax tree. A local expression normally leaves either an address or a value in `RAX`. A one-bit state, `lv`, distinguishes the two. The code generator reads it when an expression is assigned to, dereferenced, or passed to another operator.

The current source compiles under C99 with GCC. Once that executable exists, it compiles `sccc.c` into a second compiler. The second compiler compiles `sccc.c` into a third. The build checks the two resulting binaries with `cmp`. The test suite then uses both the host-built and the self-compiled binaries to compile and run programs.

2. Structure

The compiler is one C program. The large divisions in `sccc.c` are buffers and utility functions, tokenization, types, machine-code output, preprocessing, parsing, initialization, statements and functions, output-file construction, and the command-line driver. The source order roughly follows that sequence, with small forward declarations where the parser is mutually recursive.

The central compilation state is `Ctx`. It contains the current source stack and token, include paths and command-line definitions, macro state, lexical scopes, the function being compiled,

the text and data buffers, and a list of relocations. One `Ctx` is used for all source files on the command line. At the end of a translation unit, the preprocessor state and local scopes are cleared; the global definitions and output buffers remain.

```
compile_unit(ctx, src, filename, 0);
/* ... the other input files ... */
compile_unit(ctx, builtin_libc_src(), "<libc>", 1);
write_elf(ctx, output);
```

These calls show the order of the main operation. The actual `main` routine creates the context, installs types and keywords, and, for a hosted program, emits `_start` before compiling C source. `-nostdlib` omits the embedded library; `-ffreestanding` also omits the hosted startup and lets the selected kernel entry point serve as the executable entry.

The compiler does not produce `.o` files. Symbol references remain in the compilation context until `write_elf` assigns section addresses and patches the recorded locations. That function then writes one of the supported output formats. The name `write_elf` comes from the original Linux output path; it now handles flat and bootable files as well.

2.1. Buffers and allocation

`Buf` has three fields: `data`, `cap`, and `len`. Every append checks capacity, doubles it when necessary, and copies new bytes to the end. It is used for code, read-only literals, initialized data, token text, and the parts of the ELF file. `buf_patch32_at` changes a four-byte value already present in a buffer; branch patching uses it frequently. The small routines `le16`, `le32`, and `le64` write integers in the byte order required by ELF and x86-64.

Names are interned in a hash table of 8192 buckets. Each `Str` holds the spelling, length, keyword code, and an optional macro. Two occurrences of the same identifier refer to the same interned object. Name equality can therefore use pointer equality in the common parser paths. Symbols, types, members, parameters, macro expansion frames, and labels are individually allocated with `calloc` or `malloc`; their lifetime is the compilation process.

3. The language

The source language is C with the types and constructs implemented by the parser in `sccc.c`. Its basic types are `void`, `_Bool`, `char`, `short`, `int`, `long`, `float`, and `double`, together with signed and unsigned forms of the integer types. On the target, `char` and `_Bool` occupy one byte, `short` two, `int` four, and `long` eight. Pointers occupy eight bytes. Floating-point objects occupy four or eight bytes. A declaration of `long double` is handled by the `double` type path in `declspec`; it does not introduce an extended-precision representation.

`typedef`, enumerations, pointers, arrays, functions, structures, unions, and bit-fields are part of the parser. The recognized storage-class words include `static`, `extern`, and `typedef`. `auto`, `register`, and `inline` are read and discarded by `declspec`; `const`, `volatile`, and `restrict` are likewise accepted without a separate qualified type. Consequently, these spellings do not introduce an independent qualifier-enforcement or volatile-access mechanism in the generated code.

3.1. Type records and conversions

`Type` records the kind, size, alignment, signedness, base type, array length, members, parameters, and function attributes. Derived types are made by `ptr_to`, `array_of`, and `func_type`.

The same type record is carried through expression parsing. For example, `a[i]` needs the size of the object pointed to by `a`; `p->field` needs the structure member's byte offset; and a call needs the declared parameter types.

Integer promotions are performed by `int_promote`; `arith_conv` selects the common arithmetic type when two operands differ. The resulting type controls extensions, signed or unsigned division, comparisons, and floating-point operations. A load of a signed `char` extends the sign bit into the result register, while a load of an `unsigned char` fills the upper bits with zeros. A cast to `_Bool` produces zero or one.

```
unsigned char byte = 255;
int x = byte;
unsigned long mask = 1UL << 40;
double y = (double)x / 2;
```

The value of `x` is 255. The shift uses the width of an unsigned long, and the division is floating point. These are separate machine-code cases. The type rules are therefore applied while parsing, not postponed until the executable is written.

3.2. Declarators and function types

C declarators can put a name inside several layers of pointers, arrays, and parentheses. The compiler splits the work between `declspec`, `declarator`, and `type_suffix`. `declspec` obtains the initial type and storage class; `declarator` reads pointers and a name, and `type_suffix` attaches array or function suffixes. A parenthesized declarator uses a temporary type node whose contents are filled after the outside suffix has been read.

```
int *a[4];           /* four pointers to int */
int (*f)(int);       /* pointer to a function */
int (*table[2])(int); /* array of function pointers */
```

Functions have parameter records and a result type. A declaration with an empty parameter list sets an `oldstyle` flag. A declaration with `void` gives an empty explicit parameter list. `...` sets the variadic flag. Array and function parameters are converted to pointers when their parameter types are formed. The parser checks argument counts for prototyped, non-variadic functions.

3.3. Pointers and arrays

An array type retains its element size and declared length. In value contexts an array yields its address; subscripting adds an index multiplied by the size of the element. The parser also accepts the reversed C form `i[a]`, since an integer and a pointer may appear on opposite sides of the subscript calculation. Pointer subtraction divides a byte difference by the pointed-to element size.

```
struct Pair { int left, right; };
struct Pair v[3];
struct Pair *p = v;
p[2].right = 17;
```

The address used by the store is `p + 2 * sizeof(struct Pair) + offsetof(right)`. The compiler gets both numbers from the type records. `&`, `*`, `.`, and `->` operate on addresses

before `rval` converts an lvalue into its stored value. Assigning to an array or function is rejected; so is taking the address of a bit-field.

3.4. Structures, unions, and bit-fields

Structure members are laid out in declaration order with alignment between fields. The final structure size is rounded to its maximum member alignment. Union members start at offset zero, and the union size is rounded up from its largest member. A tag may first name an incomplete type and later supply its members. `Member` records contain names, types, offsets, and links to the following member.

```
struct Header {
    unsigned kind : 3;
    unsigned ready : 1;
    unsigned length : 12;
    char name[8];
};
```

Bit-fields are represented by their storage type, width, and bit position. The generator uses shifts and masks when reading or updating them. Signed fields are extended according to their width. An unnamed field can occupy bits without introducing a member name, and a zero-width field advances the packing position to the next storage unit. Anonymous structure and union members are folded into the outer member list with adjusted offsets.

A structure assignment copies bytes between addresses. The code generator uses the x86 string-copy instruction sequence `rep movsb` for this operation. Functions can return a structure, and the call mechanism provides storage for the returned object. These cases use the same size and member information that the declaration parser established.

3.5. Expressions

The expression parser uses precedence climbing. `binop_prec` gives the priority of arithmetic, comparison, bitwise, and logical operators. `parse_binary` reads the next tighter expression and then consumes operators of the requested precedence. Assignment is parsed separately, so it associates from right to left. The comma operator is handled by `parse_expr`, and the conditional operator by `parse_cond`.

```
int result = 2 + 3 * 4;
int flag = result > 10 && result < 20;
int chosen = flag ? result : 0;
```

Multiplication is emitted before addition. Comparisons produce integer truth values. `&&` and `||` use conditional jumps to skip the right operand when the left operand determines the result. This is a different code path from bitwise `&` and `|`, which evaluate both operands and combine their bits. The conditional operator also generates two control-flow paths and patches a jump where they meet.

The source handles prefix and postfix `++` and `--`, compound assignments, casts, `sizeof`, pointer arithmetic, and direct or indirect calls. Signed comparisons and unsigned comparisons select different condition codes. Integer division uses `idiv` for signed values and `div` for unsigned values. Floating-point arithmetic uses SSE instructions and separate integer/float conversion routines. The bit pattern of a floating value is carried through ordinary

expression storage in a general-purpose register; SSE registers are used while arithmetic or comparison is performed.

4. Preprocessing and tokenization

The lexer walks the source one character at a time. It skips spaces and both C comment forms, joins backslash-newline continuations, and tracks line numbers. It recognizes identifiers, numeric constants, strings, character constants, and punctuation, including multi-character operators such as `>>=`, `->`, `&&`, and `##`. A token contains a kind, integer or floating value, text, source file and line, and flags for spacing and beginning-of-line status.

Preprocessing is integrated with token retrieval. A `Src` frame is either a source file with a lexer or a sequence of tokens produced by a macro expansion. Each frame points to its parent. `raw_next` obtains the next token from this stack; `expand_next` checks identifiers for macro expansion; `pp_next` passes the result to the parser. Includes and macro expansions use the same stack and naturally return to the previous input when exhausted.

4.1. Includes and conditionals

Quoted include names are searched relative to the including file, then through the `-I` paths. Angle-bracket includes use include paths and the built-in header table. Absolute include names can be read directly. `#pragma once` records a path so a later include of that path can be skipped. Include nesting is checked against a depth of 64 file frames.

The conditional directives are `#if`, `#ifdef`, `#ifndef`, `#elif`, `#else`, and `#endif`. The preprocessor maintains a conditional stack and skips inactive source groups without presenting their tokens to the C parser. `pp_eval` expands the tokens of a `#if` expression and uses the constant-expression parser to evaluate them. `defined` is handled while the expression is prepared. The compiler also recognizes `#undef`, `#error`, `#warning`, `#line`, and `#pragma`.

```
#ifndef BUFFER_SIZE
#define BUFFER_SIZE 1024
#endif
#if BUFFER_SIZE > 512
int large_buffer(void) { return 1; }
#endif
```

Conditional compilation has to run before declarations are analyzed: if a branch is inactive, the parser never sees its declarations or errors. Source file and line information follows the token through expansions so errors can be reported at their origin.

4.2. Macros

A `Macro` stores its name, parameter names, replacement tokens, and flags for function-like and variadic forms. The expansion routine collects actual arguments while counting nested parentheses. Arguments used normally are expanded before substitution. Arguments adjacent to `##` use their original token spellings. The `#` operator creates a string token, and `##` concatenates two spellings and lexes the result into a new token.

```
#define CAT(a, b) a ## b
#define STR(x) #x
#define TWICE(x) ((x) + (x))
int CAT(total, _count) = TWICE(4);
```

A macro that is currently being expanded is marked busy, preventing it from immediately expanding itself again. An empty macro argument may be represented temporarily by a placemarker token during concatenation. Variadic expansion supports `__VA_ARGS__`, including the comma-elision case used by the implementation. Predefined macros include `__FILE__`, `__LINE__`, `__STDC__`, platform names, and the target size constants. The builtin header and libc sources go through the same preprocessor as the user's files.

5. Parsing, names, and storage

The compiler keeps lexical scope with a linked list of `Sym` records and a nesting depth. `enter_scope` increments the depth. `leave_scope` drops the symbols introduced in that scope, exposing names that were shadowed. `Tag` records provide a separate list for structure and union tags. Enumeration constants use symbol entries with fixed integer values. A `typedef` name refers to a type rather than storage.

Global definitions have another representation. Their symbol entries include a section, an offset, a definition flag, and linkage information. `get_global` finds or creates such entries. A declaration seen in one file can therefore supply the target for a reference emitted in another file. Local variables instead use stack offsets recorded in their symbols.

5.1. Declarations are compiled as they are read

`parse_toplevel` reads one declaration at a time. A function declaration installs its type and name. A function definition continues into `parse_func_body`, which immediately emits the prologue, parameters, statements, and epilogue. A global initializer writes bytes to the data buffer or records a relocation if its value is an address. Declarations with no initializer normally reserve zero-initialized storage.

Inside a block, the parser recognizes declarations alongside statements. Automatic objects receive frame offsets through `alloc_local`. Their initializers generate instructions into the function body. The compiler also implements local `static` objects, which receive global storage while retaining the local name used to address them.

Consider a definition that comes after its caller:

```
int calculate(int);
int main(void) { return calculate(7); }
int calculate(int n) { return n * n; }
```

When `main` is parsed, `calculate` already has a function symbol, but not a code address. The call instruction receives a relocation record. When `calculate` is defined, the symbol records its position in the text section. `write_elf` uses that position to resolve the earlier call.

5.2. Initializers and constant expressions

Global initializers and local initializers are processed differently. For globals, integral and floating constants are converted into bytes in `.data`. An address initializer can write a placeholder together with an absolute relocation. For locals, the initializer evaluates at run time and stores through an address in the current stack frame. Array and structure initializers first clear the aggregate storage and then fill the specified fields.

The recursive initializer routines handle braces, string literals, arrays of unknown length, nested records, and designated initializers. An array without an explicit length obtains its

length from its initializer. Designators such as `[3] =` or `.name =` select an element or member before parsing its value.

```
struct Item { int id; char code[4]; };
struct Item items[] = {
    [0] = { .id = 3, .code = "AB" },
    [2] = { .id = 9 }
};
```

The omitted array element is zero-initialized, and the unknown array length becomes three. The distinction between expressions executable at run time and expressions required to be constant appears here, in enumeration values, in `case` labels, and in array lengths. `const_expr` evaluates the integer form; separate routines evaluate floating constants for data initializers.

6. Machine-code generation

The backend writes x86-64 machine instructions directly. `Code` contains a byte buffer; the short routines beginning with `emit_` append opcode bytes, prefixes, ModR/M bytes, displacements, or immediate values. The small `C8`, `C32`, and `C64` functions are the final byte-writing operations. There is no textual assembly between expression parsing and the executable.

For example, loading a 64-bit constant into `RAX` writes the `REX.W` prefix, the immediate-move opcode, and eight bytes of data:

```
static void emit_mov_rax_imm64(Code *c, uint64_t imm) {
    C8(c, 0x48);
    C8(c, 0xB8);
    C64(c, imm);
}
```

There are analogous helpers for stack operations, integer arithmetic, comparisons, shifts, indirect calls, and floating-point instructions. The generated program is consequently built from a small collection of explicit encodings rather than from an instruction-selection table.

6.1. Values and addresses

The `lv` state tells whether an expression has produced an address. An identifier denoting a local variable emits its frame-relative address; a global variable emits a RIP-relative address with a relocation. `rval` then loads the object if a value is needed. Arrays yield pointers to their first elements, and a function designator yields a callable address. The same address machinery handles `&a`, `*p`, subscripts, and structure members.

```
int v = 10;
int *p = &v;
*p = *p + 1;
```

For the assignment, the left-hand dereference leaves a destination address. The compiler pushes it, computes the value on the right, restores the destination into another register, and chooses a store instruction using the size of the pointed-to type. A four-byte store and a one-byte store have different encodings; a bit-field store also reads the old storage unit, changes selected bits, and writes the unit back.

Most integer operations accumulate their result in `RAX`. For a two-operand expression the left value is pushed, the right value is computed, and the left is popped into `RCX`. Arithmetic then uses the pair of registers. `gen_push` and `gen_pop` maintain a count of temporary stack slots. The code generator uses it when making calls, because the stack must be aligned at the call site.

6.2. Floating-point instructions

The parser keeps the type of a floating expression even when its bytes reside in `RAX`. For an operation, helper routines transfer the bit pattern into SSE registers, perform the requested instruction in single or double precision, and move the result back. Integer-to-float, float-to-integer, and float-to-float conversions have distinct paths. Floating comparison uses the SSE unordered comparison instructions and examines the condition flags, including the behavior of NaN.

The floating-point tests exercise mixed arithmetic, global constants, arrays of doubles, casts, comparisons, format printing, and mathematical functions from the embedded library. They also include a loop that adds `0.1` repeatedly, whose comparison with `1.0` shows the ordinary consequences of binary floating-point representation.

6.3. Branches and statements

A conditional branch can be emitted before its target has a position in the text buffer. `emit_jz_placeholder` and its relatives leave four zero bytes for a relative displacement. `patch_jmp_to` later replaces them with the difference between the target and the address following the instruction. Each `if` and `else` generates one or two such branches.

`while` records the condition position and emits a backward jump from the bottom of its body. `do` places its test after the body. `for` has condition, step, and body positions; the emitted code can jump over the step on the first iteration and return to it after the body. `break` and `continue` are stored in patch lists belonging to the active loop. A nested `switch` uses a separate kind of loop context so `continue` can still find the enclosing iteration statement.

`switch` first saves its controlling value. The case labels record positions in the body. After the body has been emitted, the compiler writes a chain of comparisons and jumps to those positions, with a final jump to `default` when present. `goto` names are stored in a function-local label list. A forward goto adds a patch to its label; defining the label resolves the pending jumps. Undefined and duplicate labels are diagnosed.

7. Functions and calls

A function begins with `push rbp`, establishes `rbp` as its frame pointer, and reserves a frame whose size is patched after the body has been parsed. Parameters in registers are copied into slots near the top of the frame. Locals and compiler-generated temporaries are assigned negative offsets from `rbp`. The final epilogue uses `leave` and `ret`, restoring the stack to its state at entry.

7.1. Argument placement

Generated ordinary calls use six general-purpose argument registers, in the order `RDI`, `RSI`, `RDX`, `RCX`, `R8`, and `R9`. Further argument slots are passed on the stack. Arguments are evaluated and saved, then copied to their calling locations. Padding is inserted as needed for 16-byte

stack alignment. A direct call uses a relative `call` instruction and a symbol relocation. A call through a function pointer loads its address and uses an indirect call through `R11`.

The compiler's generated convention handles scalar arguments in eight-byte slots, including floating-point bit patterns. Arithmetic inside a function still uses SSE; its calls pass the bits through the general-purpose slots. Structure parameters are copied into the callee's local frame from addresses supplied by the caller. A structure return passes a hidden destination address in `RDI`, leaving five ordinary register positions. These rules are shared by the compiled program and the bundled libc. They are not the full System V x86-64 aggregate and SSE argument classification used by separately compiled system libraries.

7.2. Variadic functions and system calls

A variadic function saves the six register argument slots in its frame. The built-in operation `__builtin_va_start` initializes a `va_list`-like record with the position of the next register argument, the beginning of stack arguments, and the register-save area. The library's `__va_arg_ptr` obtains the next value from that record. This supports functions such as `printf`, which read values according to their format strings.

Linux system calls do not use the ordinary function-call register list. The compiler recognizes `__syscall` and places the call number in `RAX`, with syscall arguments in `RDI`, `RSI`, `RDX`, `R10`, `R8`, and `R9`. It then emits the two-byte `syscall` instruction. The distinction appears directly in the implementation: `parse_call` generates C calls, while `parse_builtin_syscall` generates the Linux entry sequence.

8. The embedded C library

The files in `lib/include` define the public interfaces, and `lib/libc.c` implements them. `tools/embed.sh` converts these files into C string arrays in `builtin.h`. The host builds `sccc.c` with that header. When the resulting compiler encounters `#include <stdio.h>` or another embedded header, it supplies the corresponding stored text. When user compilation finishes, it compiles the embedded `libc.c` as another C translation unit unless `-nostdlib` was specified.

The library provides system-call wrappers, memory allocation, strings, character classification, file and formatted I/O, conversions, and mathematical routines. Its wrappers use the Linux syscall numbers for x86-64, including `read`, `write`, `open`, `close`, `lseek`, `mmap`, and `exit_group`. A negative syscall return value is converted to `-1` and stored in `errno` when appropriate.

The allocator obtains memory from `mmap`. Small allocations are grouped into size classes with free lists, and larger allocations obtain their own mapped blocks. `malloc`, `free`, and `realloc` are therefore available to programs compiled by SCCC. This matters for the compiler source itself: it makes extensive use of dynamically grown buffers and individually allocated symbols and types.

The I/O routines build on file descriptors. The library implements stream operations such as `fopen`, `fread`, `fwrite`, `fseek`, and `fprintf`; the tests use actual files as well as standard input and output. Formatted output covers integer formats and floating-point formats. The library is compiled by the same compiler as the program, so its functions follow the calling rules described in the preceding section.

A program can also be compiled with `-nostdlib`. In this mode, no library functions are added and no library startup call is emitted. The program must supply every function it references. The generated entry still calls `main` and terminates through the Linux exit syscall if no `exit` definition was supplied.

9. The ELF executable

For a default hosted build, `write_elf` gives final locations to text, read-only constants, initialized data, and BSS. The executable begins at the virtual base address `0x400000`. The ELF header is 64 bytes, followed by two 56-byte program headers. The generated code begins after these 176 bytes. `.rodata` follows the text at a 16-byte boundary; `.data` begins at a page boundary; `.bss` follows initialized data in memory.

The first loadable segment is readable and executable and covers the headers, machine code, and read-only data. The second is readable and writable and contains initialized data followed by the zero-filled BSS region. ELF program headers tell Linux how to map those areas when the process starts. Section headers are also written for `.text`, `.rodata`, `.data`, `.bss`, `.symtab`, `.strtab`, and `.shstrtab`. The symbol table contains `_start` and defined C symbols, with local and global bindings.

9.1. Resolving addresses

Relocations have two forms. `R_PC32` holds a signed 32-bit displacement relative to the end of an instruction, used for calls and RIP-relative addresses. `R_ABS64` holds an eight-byte absolute address, used for initialized pointers. Each relocation stores its position, symbol or section, and addend.

For a relative call the calculation is:

```
rel = target_address - (instruction_address + 4);
```

Here `instruction_address` means the address of the four-byte displacement field. The extra four advances to the following instruction. The writer checks that the result fits in a signed 32-bit value, then patches those bytes. For an absolute relocation, it writes the full 64-bit target address into the data section. An unresolved symbol produces an error before the file is finished.

9.2. Process startup

In a hosted executable the first emitted instructions belong to `_start`, not to `main`. They read `argc`, `argv`, and the environment address from the initial process stack, align the stack, and, when the library is present, initialize `environ` and call `__libc_init`. They then call `main`. Its integer result becomes the exit status passed to `exit`, or, without that library function, to the `exit_group` syscall.

The Linux ELF output is written with mode `0755`. Its binary bytes are fully determined by the input, emitted encodings, symbol layout, and section placement. There is no timestamp inserted into the ELF image by the writer.

10. Freestanding compilation

`-ffreestanding` changes how a compilation begins and ends. It sets `freestanding` and `nostdlib` in `Ctx`, so `main` is not required, `_start` is not generated, and the embedded C library is not compiled. The predefined macro `__STDC_HOSTED__` becomes 0. In this mode the

program supplies its own entry function and whatever services it needs. A common entry is `kmain`, chosen with `-e kmain`.

```
./sccc -ffreestanding -mgeneral-regs-only \
  -oformat img -e kmain baremetal/hello.c -o hello.img
```

This command compiles an ordinary C source file. The C parser, type system, initialization rules, symbol table, and x86-64 emitter are the same ones used for Linux programs. The difference lies in startup and output. The entry symbol is resolved after all source files have been read, and the disk-image writer puts its address into the generated boot sector. The kernel does not enter through a C runtime; the boot code calls the selected function directly.

`-mgeneral-regs-only` makes a use of `float` or `double` an error. It is checked by `declspec` and by the floating-point expression paths, and it may be used with `-nostdlib` or `-ffreestanding`. This option is also required for interrupt functions. The generated kernel therefore uses the integer registers without depending on FPU or SSE state management during an interrupt. There is no new dialect of C for the kernel: pointers, structures, arrays, function calls, and control flow use the same compiler machinery.

Three options determine kernel placement. `-oformat` chooses `elf`, `bin`, `img`, or `multiboot`; `-Ttext` selects the address at which `.text` executes; `-e` names the entry function. The default text address for a BIOS image is `0x100000`, and the default for a Multiboot image is `0x101000`. The corresponding addresses are absolute runtime addresses, not offsets in the disk file. For a flat binary, the default text address is zero, and its entry must be the first byte of the text. The writer checks that condition.

11. Flat binaries and boot images

11.1. Section layout

The layout code in `write_elf` is shared by all four output formats. `.text` contains emitted instructions, `.rodata` contains strings and other read-only constants, `.data` holds initialized objects, and `.bss` reserves zero-filled memory. References between these sections are resolved before the output bytes are written. In the flat format the bytes of `.text`, `.rodata`, and `.data` appear consecutively with alignment padding. `.bss` occupies no file bytes; its extent is included in the address calculations used by the boot loaders.

```
./sccc -ffreestanding -oformat bin -Ttext 0x100000 \
  -e _start kernel.c -o kernel.bin
```

Here `kernel.c` defines `_start` as its first function. In the flat case `-Ttext` describes where the loader will place the first instruction. The file contains no ELF header and no loader of its own. A kernel using a later entry address is assembled by choosing an output format that records that address. `-oformat img` and `-oformat multiboot` both place their own startup code before the kernel and arrange to call the selected entry point.

11.2. The BIOS disk image

`write_image` prepends a 512-byte boot sector to the kernel image. `boot_sector` constructs that sector from x86 instruction bytes with calls to `C8`, `C16`, `C32`, and `C64`, just as the compiler's main emitter constructs 64-bit instructions. Its execution begins at `0x7c00` in the mode established by the PC BIOS. The final two bytes of the sector are `0x55` and `0xaa`, the BIOS boot signature. The kernel itself starts at the next disk sector.

Before loading the kernel, the boot sector requests the BIOS E820 memory map with `int 0x15`. It stores the entry count at physical address `0x5000`, followed by 24-byte entries beginning at `0x5008`. Each entry gives a base address, a length, a type, and extended attributes. This table is passed to C code later; the `memory` kernel reads it to decide which physical page frames are available.

The sector reads the kernel from disk with the extended `int 0x13` interface, using a disk address packet. The initial load address is `0x10000`, below the 1 MB boundary where the firmware interface can reach it. Reads proceed in batches of at most 64 sectors. A disk-read failure prints an `E` through the BIOS video interrupt and halts. The image writer measures the kernel, rounds its file size to whole sectors, and records the count in the boot sector. It rejects kernels larger than 588800 bytes.

After the read, the sector enables the A20 line through port `0x92` and constructs page tables at physical addresses `0x1000` through `0x3fff`. A PML4 entry points to a page-directory-pointer table, whose first entry points to a page directory containing 512 entries of 2 MB each. The result is an identity mapping of the first gigabyte. The setup loads `cr3`, enables PAE in `cr4`, sets the long-mode enable bit in `EFER`, loads a GDT, and enables paging in `cr0`. A far jump completes the transition into 64-bit long mode.

Now the boot code copies the kernel from its temporary load buffer to the `-Ttext` address, clears `.bss`, sets `rsp` to `0x80000`, places `0x5000` in `rdi`, and calls the entry function. The C entry therefore receives a pointer to the memory-map record without calling the BIOS itself. The loader leaves interrupts disabled. The kernel decides when to construct an IDT and enable external interrupts.

The image is padded to a cylinder-sized multiple of 516096 bytes, using 16 heads and 63 sectors per track. The boot image is therefore a complete raw PC disk image. QEMU and Bochs boot it through their BIOS disk paths.

11.3. The Multiboot image

The Multiboot output has a different first page. `write_multiboot` writes the Multiboot 1 magic value `0x1badb002`, flags, checksum, and explicit load addresses, followed by a trampoline that starts in 32-bit protected mode. The next page begins the x86-64 kernel at the text address, normally `0x101000`. The text address must be page aligned. The image can be loaded by GRUB or by QEMU's `-kernel` option.

The Multiboot trampoline receives the bootloader's registers and saves them while it constructs identity-mapped page tables. Its page tables and a 16 KB stack occupy space after the kernel's BSS. It sets `cr3`, `cr4`, `EFER`, and `cr0`, loads a small GDT, and jumps into long mode. It then passes the address of the Multiboot information structure in `rdi` and the magic value `0x2badb002` in `rsi`, and calls the C entry. The BIOS image and the Multiboot image reach the same kind of 64-bit kernel function by different boot paths.

```
./sccc -ffreestanding -mgeneral-regs-only \
  -oformat multiboot -e kmain baremetal/hello.c -o hello.mb
qemu-system-x86_64 -kernel hello.mb -serial stdio
```

The Multiboot header and the transition instructions are emitted by SCCC. There is no separate assembly source for either boot path and no post-processing step that converts a host executable into a kernel image.

12. Machine instructions and interrupts

12.1. Hardware built-ins

The compiler recognizes machine operations by name while parsing a call. Their encodings are described in `mach_builtins`, an array of records containing the name, argument count, argument registers, result type, instruction length, and bytes to emit. Arguments are parsed as normal C expressions, checked as integers or pointers, and moved into the required registers. The emitter then appends the bytes. For example, `__outb` places the port in `dx`, the value in `al`, and emits opcode `0xee`. `__inb` emits `in al, dx` followed by an extension that produces an unsigned-byte result in `eax`.

```
__outb(0x3F8, 'A');
unsigned char status = __inb(0x3FD);
__cli();
__lidt(descriptor);
__sti();
__hlt();
```

The built-ins cover byte, word, and double-word port I/O; `cli`, `sti`, `hlt`, and `pause`; loading the GDT and IDT; loading the task register; reading and writing `cr0`, `cr2`, `cr3`, and `cr4`; reading and writing model-specific registers; invalidating a TLB entry with `invlpg`; querying `cpuid`; and reading `rdtsc`. The `__cpuid` operation stores four 32-bit registers into the caller's result array. `__rdmsr` and `__rdtsc` combine `edx:eax` into a 64-bit C result.

There is also a distinct `__syscall` built-in for Linux, whose arguments follow the Linux syscall register order. Kernel examples use the hardware built-ins to reach I/O ports and control registers, without relying on inline assembly. The compiler emits precisely the processor operations named in the source.

12.2. Interrupt functions

An interrupt gate cannot enter a function through an ordinary C `call`. The processor has already pushed an interrupt-return frame, and the exit instruction must be `iretq`, not `ret`. A fault that supplies an error code also changes the location of that frame. SCCC handles both cases with the `interrupt` function attribute.

```
struct frame {
    unsigned long ip, cs, flags, sp, ss;
};

__attribute__((interrupt))
void on_timer(struct frame *f) {
    (void)f;
    ticks++;
    __outb(0x20, 0x20);
}
```

`emit_interrupt_entry` writes a wrapper at the beginning of an attributed function. It first clears the direction flag, then saves `rax`, `rcx`, `rdx`, `rsi`, `rdi`, and `r8` through `r11`. It aligns the stack for a call into the regular C function body and passes a pointer to the processor's interrupt frame as the first argument. If the handler has a second parameter, the wrapper reads the hardware error code and passes it as an `unsigned long`. On return it restores the saved registers, removes the error code where present, and executes `iretq`.

The compiler checks the function signature: an interrupt function returns `void` and has either a frame-pointer argument or a frame pointer followed by one 64-bit integer error-code argument. It also requires `-mgeneral-regs-only`. The ordinary function prologue and body follow the generated wrapper, so an interrupt handler can use local variables, C expressions, and calls to other C functions. `emit_interrupt_entry` makes the ABI boundary explicit in the generated instructions.

12.3. Descriptor tables and devices

`baremetal/kernel.h` supplies the small amount of code shared by the example kernels. It writes a 16-byte IDT gate as two 64-bit words, constructs a 10-byte IDTR descriptor, and loads it with `__lidt`. `pic_init` remaps the master and slave 8259 interrupt controllers, normally placing hardware IRQs at vectors 32 and above. The same header initializes the first serial port at `0x3f8`, waits for transmitter status at `0x3fd`, and prints characters with `__outb`.

The header also contains a `finish` routine. It waits for serial output to drain, writes `0x10` to port `0xf4` for QEMU's ISA debug-exit device, and sends `Shutdown` to Bochs through port `0x8900`. These ports provide a programmatic end to kernel tests. The remaining code path disables interrupts and halts.

13. Kernel programs

The `baremetal` directory contains five kernel programs written in C. Each uses the disk image output, starts through the generated boot sector, and runs without the Linux runtime. Their output goes through COM1; `hello` also writes directly to VGA text memory at physical address `0xb8000`.

The `hello` kernel initializes the serial port, writes a line of text, fills the 80-by-25 VGA character cells, and verifies that the character-and-attribute words can be read back. It prints the result over serial before stopping the emulator. The kernel does not call `printf`. Its output routine is a loop over I/O-port writes in `kernel.h`.

The `timer` kernel constructs an IDT entry for vector 32, reprograms the PIC, and sets the programmable interval timer by writing its control word to port `0x43` and the divisor to port `0x40`. It enables interrupts with `__sti` and halts between ticks with `__hlt`. The interrupt function increments a counter and acknowledges the PIC by writing `0x20` to port `0x20`. The main loop prints every tenth tick up to 50. It also reads the timestamp counter before and after the run.

The `exceptions` kernel installs handlers for divide error (#DE), breakpoint (#BP), and page fault (#PF). It enters the breakpoint vector with a byte sequence containing `int3`, then performs an integer division by zero and changes the saved instruction pointer in the exception frame to resume execution. The page-fault handler reads `cr2` to find the faulting address, examines `cr3` to locate the page tables, inserts a second identity-mapped gigabyte, and invalidates the faulting translation with `__invlpg`. The test checks both the supplied error code and the address that caused the fault. It also verifies that a local 64-bit value survives the exception sequence.

The `keyboard` kernel handles PS/2 keyboard interrupts on vector 33. Its handler reads a scancode from port `0x60`, puts it into a 64-entry ring buffer, and acknowledges the PIC. The main loop consumes the buffer, tracks Shift, converts key presses through a table, and echoes characters over serial until Enter. The QEMU test sends keys through its monitor.

The Bochs build defines `INJECT` and places the corresponding scancodes into the controller through ports `0x64` and `0x60`.

The `memory` kernel receives the E820 map passed by the BIOS boot sector. It creates a bitmap covering the first gigabyte in 4096-byte page frames, marks only usable memory above 2 MB available, and allocates and releases frames while checking their contents. It also finds a consecutive 256-frame run and uses it as a one-megabyte heap. The heap keeps a linked list of blocks, splits a block for allocation, and coalesces adjacent free blocks on release. A test performs 3000 pseudorandom allocations and frees, checking data and 16-byte alignment. Both the page-frame and heap tests finish with their free counts restored.

The five programs exercise different parts of the compiler while running on the processor directly: volatile memory accesses, integer types and pointer arithmetic, hardware operations, function wrappers, control-flow patching, constant data, global initialization, and structure layouts. The `baremetal/run.sh` suite compiles each kernel, boots it in QEMU and Bochs, and compares its serial output with the checked-in `.out` file. QEMU completion is also checked through the debug-exit status; Bochs completion is checked through its shutdown request. The source of the kernels and their expected output live together in the repository.

```
make baremetal
# or, as part of the complete test suite:
make check
```

The boot sector, Multiboot trampoline, hardware built-ins, and interrupt wrappers are all generated by the compiler itself. The kernel source uses C declarations and statements for its work after the processor reaches 64-bit mode.

14. Multiple source files

SCCC accepts multiple C files on one command line. Each file has its own preprocessing run and file-scope names, while definitions destined for the executable share the same context. A function defined in one input can satisfy a call emitted for a declaration in another. The compiler resolves these references after compiling all inputs and the embedded library.

```
./sccc examples/modules.c examples/modules_part2.c -o modules
./modules
```

The `modules` example exercises a declaration in a header and definitions in separate source files. The multi-file language test also checks globals, private functions, common library names, and file I/O. The library is compiled after user sources, allowing user definitions to take precedence where the implementation's global-symbol rules permit it.

There is no separate object-file interface and no archive search. Supplying all sources in one invocation is the compilation model. The `-o` option chooses the executable name; `-I` adds header directories; `-D` supplies command-line macro definitions; `-nostdlib` omits the built-in library. Without `-o`, the output is `a.out`. The command line accepts up to 64 source inputs, with fixed-size arrays for include directories and `-D` definitions.

15. Self compilation and testing

The bootstrap rule is short enough to give in full:

```
./sccc sccc.c -o sccc2
./sccc2 sccc.c -o sccc3
cmp sccc2 sccc3
```

The host compiler builds the first `sccc` from the repository source. The second generation is produced by that executable. The third is produced by the second. The comparison checks that the last two generations emit exactly the same executable bytes for the same compiler source. The Makefile prints `bootstrap verified: sccc2 == sccc3` when the comparison succeeds.

The hosted compiler has two main kinds of tests. `test_sccc.c` writes small C programs to temporary files, compiles them, runs the results, and compares exit statuses or expected compilation failures. The feature tests in `tests/` compile complete programs and compare their printed output against checked-in `.out` files when such files exist. They cover functions, pointer operations, arithmetic, statements, structures, globals, I/O, the preprocessor, scopes, floating-point arithmetic, initialization, bit-fields, and multiple translation units.

The `examples/` directory holds larger programs. Among them are a parser for Lisp-like input, SHA-256, sorting, a maze, a small language interpreter, numerical calculations, and terminal games. `examples/run.sh` compiles each program, runs it with the supplied inputs and arguments, and compares its output with the stored result. `make check` runs the C driver, feature suite, and examples against both `sccc` and `sccc2`, and invokes `baremetal/run.sh` for the kernel tests. `make baremetal` runs the kernel suite separately. `make fuzz` runs a generated-expression test separately.

For the archive used in these notes, `make check` completed with 96 passing C-driver cases, 16 passing feature programs, and 47 passing examples for **each** compiler generation. The two bootstrap files also compared equal. The scripts report zero failures in all three groups. The complete output can be reproduced from the Makefile rather than from a separately maintained list of claimed successes.

There are two different properties under test. Byte identity checks reproduction of the compiler binary. Running programs checks the behavior of generated executables. A compiler can reproduce an error consistently, so the second group of tests remains part of the bootstrap procedure. Both generations are asked to translate other source code, not only their own.

16. Source size and executable size

The compiler source contains 4482 lines of C. The embedded library is written in 1468 lines in `lib/libc.c`; its public declarations live in the separate header files. The C test driver is another 463 lines. `builtin.h` is generated from the library and headers, so its size is a consequence of the embedding process, not an independently maintained implementation.

In the bootstrap run described above, the host-built `sccc` executable occupied 232208 bytes. Both self-compiled stages occupied 413968 bytes. The host compiler was invoked with `-O2`; SCCC emits its own code directly, without a global optimization pass. Those sizes include different machine-code sequences for the same compiler source. The equal sizes of `sccc2` and `sccc3` are accompanied by their byte-for-byte equality, checked by `cmp`.

The work done by the compiler is concentrated in two places. Most source constructs cause machine instructions to be emitted as soon as the parser recognizes them. After all input

has been translated, `write_elf` walks the relocation list, fixes addresses, and copies the sections into the selected output file. `write_image` and `write_multiboot` add their generated bootstrap code to the flat kernel bytes. The implementation has no object-file readback pass or separate loader program.

17. Implementation notes

A few details determine the shape of this compiler. First, expression code is generated during parsing. Constant expressions are handled by separate evaluators because they need values before run time. Second, name resolution occurs at two different moments: lexical scopes are resolved as tokens arrive, while global addresses are resolved after section layout. Third, the calling rules are implemented on both sides of every generated call, including calls made from the embedded library.

These details also explain where to look in the source. The `Type` and `Member` definitions give the target's data layout. `declspec` and `declarator` show how source declarations become those types. `parse_binary`, `parse_assign`, and `parse_stmt` show C expressions and statements becoming machine instructions. `parse_func_body` and `parse_call` give the function convention. `write_elf` completes all section layouts and fixes unresolved addresses; `boot_sector`, `write_image`, and `write_multiboot` construct bootable output. `mach_builtins` contains the hardware instruction encodings, and `emit_interrupt_entry` shows the register-save and interrupt-return sequence.

The test directories give concrete programs for the compiler's behavior. They cover bit-field initialization, variadic calls, multi-file definitions, floating-point formatting, interrupt handlers, peripheral I/O, exception frames, timers, and physical memory. The bare-metal cases exercise emitted boot code and kernel entry in addition to the language features exercised by the hosted suite.

18. Conclusion

SCCC reads C source and writes x86-64 machine code. Its preprocessor supplies tokens to a recursive-descent parser; the parser constructs types and emits instructions; the output pass assigns addresses, resolves relocations, and writes an ELF file, flat binary, BIOS disk image, or Multiboot image. Hosted executables use its embedded headers and C library. Freestanding kernels enter through generated boot code, use machine built-ins for hardware access, and handle interrupts through generated wrappers. The second and third compiler generations reproduce identical executable bytes.

The source follows the path taken by a C construct from its spelling to the bytes that execute it. A pointer subscript becomes a scaled address; a local declaration becomes a frame offset; a forward function call becomes a relocation; an interrupt function gains a register-save wrapper and an `iretq`; a BIOS image gains the instructions that bring the processor from real mode into long mode. Those operations are the compiler.

Availability. The source, library, hosted and bare-metal tests, kernel examples, and bootstrap Makefile are distributed in the SCCC project at: `https://github.com/gaidardzhiev/sccc` The compiler and library source carry the GPL-3.0-only license.